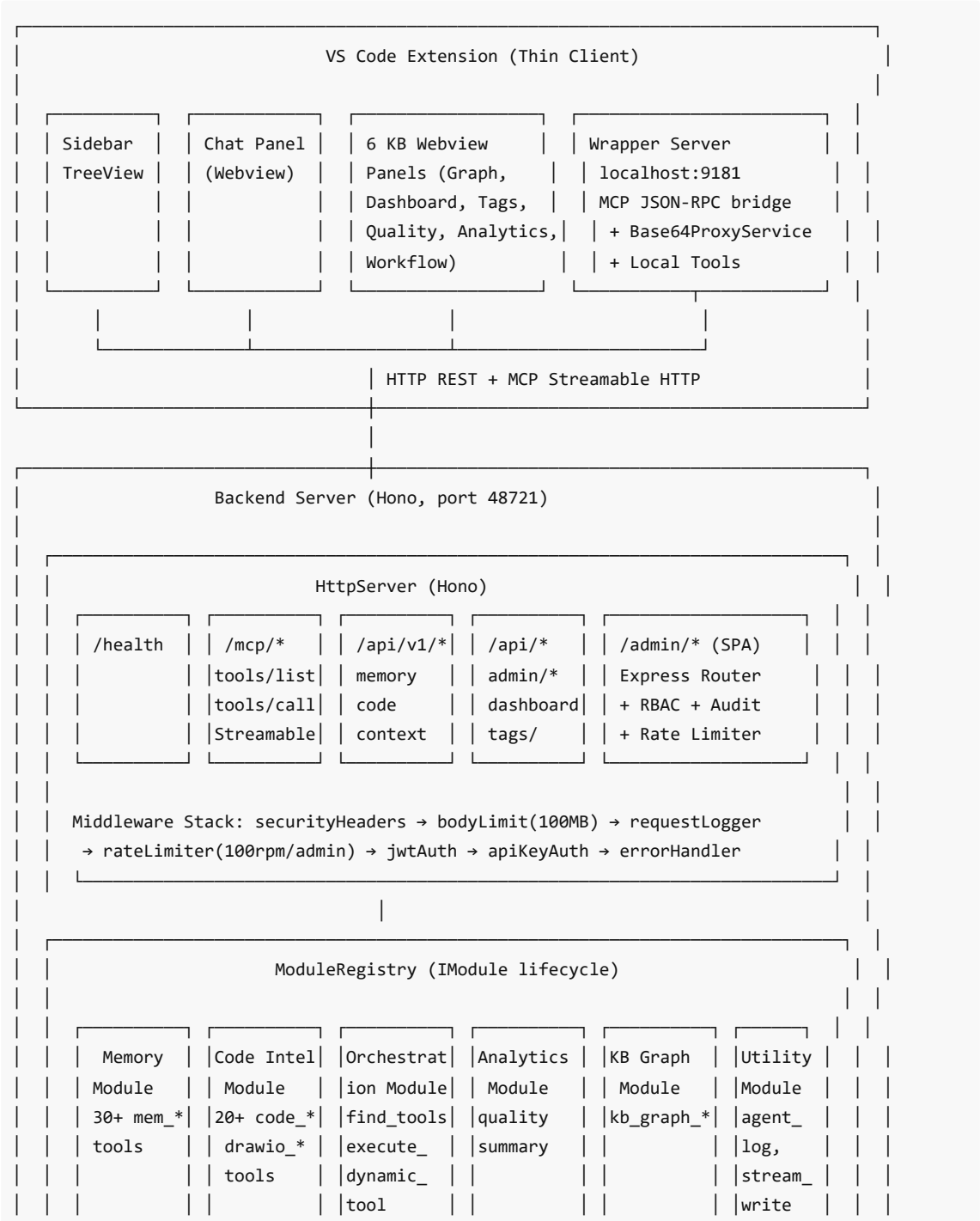


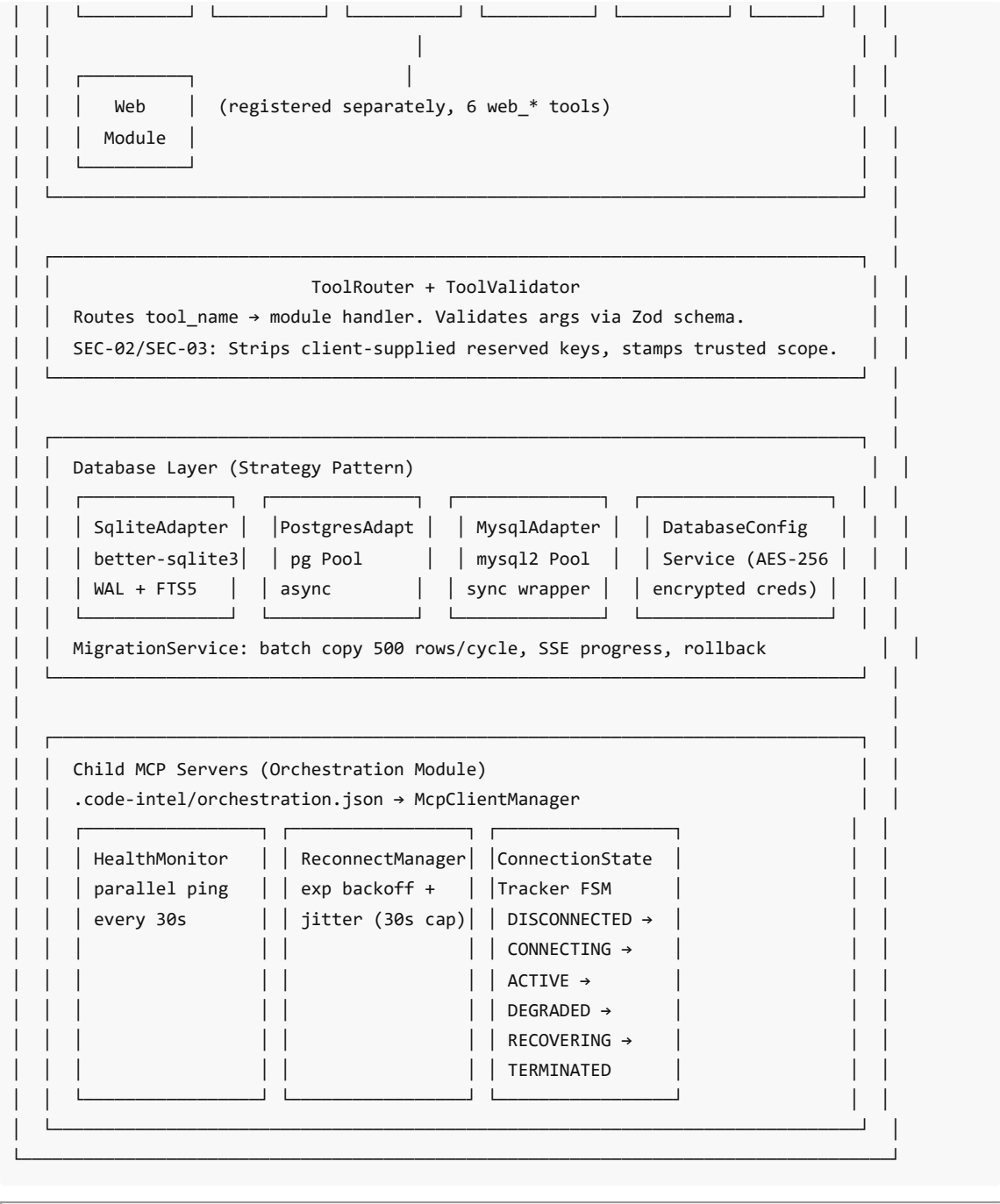
SDLC Agents 4 Enterprise — Kiến trúc chi tiết

1. Tổng quan hệ thống

SDLC Agents 4 Enterprise là một **multi-agent pipeline** tự động hoá toàn bộ vòng đời phát triển phần mềm (SDLC). Hệ thống gồm 9 AI agent chuyên biệt được điều phối bởi Scrum Master, chạy trên nền tảng **MCP (Model Context Protocol)**.

Kiến trúc tổng thể





2. Backend Server (backend/)

2.1 Entry point (src/index.ts)

Quy trình khởi động:

1. **Load config** — loadConfig() đọc từ CLI args → env vars → config.json trong .code-intel/ .Zod validate schema đầy đủ.
2. **Khởi tạo ModuleRegistry** — chứa Map<string, IModule>, quản lý vòng đời (register → initializeAll → shutdownAll).
3. **Register 6 modules**: Memory, CodeIntel, Orchestration, Analytics, KBGraph, Utility.

4. **initializeAll()** — gọi song song `module.initialize()` trên tất cả modules, dùng `Promise.allSettled()` .
5. **Ingest tools vào mcp_tools table** — lấy tất cả tool definitions từ registry + proxied tools từ orchestration module, sinh embedding vector (384-dim Float32 via ONNX), INSERT/UPDATE vào `mcp_tools` table để phục vụ `find_tools` .
6. **Start HttpServer** — tạo Hono app, mount routes, gọi `serve()` .
7. **Graceful shutdown** — bắt SIGTERM/SIGINT, gọi `server.stop()` → `registry.shutdownAll()` .

2.2 HTTP Server (`src/server/HttpServer.ts`)

Dùng **Hono** framework (TypeScript-first, lightweight). Route tree:

```
GET /health                → health.ts (status, version, uptime, modules, toolCount)
GET /mcp/tools/list         → tools.ts (CORE tools only, tiered visibility)
POST /mcp/tools/call        → tools.ts (validate args, SEC-02 stamp, route to handler)
ALL /mcp                   → mcpServer.ts (MCP Streamable HTTP transport)
POST /api/v1/memory/search  → kb-api.ts (proxy to mem_search handler + projectContext)
POST /api/v1/memory/ingest  → kb-api.ts (proxy to mem_ingest handler)
POST /api/v1/memory/ingest-file → kb-api.ts
POST /api/v1/code/search    → kb-api.ts
POST /api/v1/context/curated → kb-api.ts
GET /api/v1/admin/status    → kb-api.ts
POST /api/v1/admin/migrate-scope → kb-api.ts
GET /api/tools              → kb-api.ts (CORE only + meta tools)
POST /api/tools/find        → kb-api.ts (semantic tool search)
POST /api/tools/execute     → kb-api.ts (execute any tool by name)
GET /api/dashboard/*       → api.ts (summary, recent entries)
GET /api/kb/graph/*        → api.ts (graph nodes, edges)
GET /api/analytics/*       → api.ts (overview, timeline)
GET/POST/PUT/DELETE /api/tags/* → api.ts (tag CRUD)
GET /api/quality/*         → api.ts (scores, summary)
POST /api/index/source      → api-index.ts (index source code)
POST /api/index/document[s] → api-index.ts (index documents)
GET /api/admin/database/status → database.ts
POST /api/admin/database/* → database.ts (test-connection, migrate, switch)
GET /api/admin/me          → admin routes
ALL /api/admin/*           → admin routes (14 files: auth, config, context, KB entries,
KB graph, spatial, MCP ops, RBAC, SSE, users, etc.)
ALL /api/mcp-servers/*     → McpConfigRoutes (CRUD child server configs)
```

Middleware Stack (thứ tự áp dụng):

1. `securityHeaders` — CSP, X-Content-Type-Options, X-XSS-Protection, Referrer-Policy, Permissions-Policy, xoá X-Powered-By
2. `bodyLimit` — 100MB max request body
3. `requestLogger` — Pino structured logging (debug cho `/health`, info cho còn lại)
4. `rateLimiter` — sliding window 100 req/min/IP trên `/api/admin/*` (dev: 10.000)
5. `jwtAuth` — required trên `/api/index/*`, `/api/v1/*` . Hỗ trợ 2 loại token: JWT (HS256) + admin session token (opaque hex, validate qua sessions table). Anonymous fallback nếu không có header và `CODE_INTEL_REQUIRE_AUTH` không bật. Gắn `projectContext` vào Hono context.
6. `apiKeyAuth` — required trên `/api/tags/*`, `/mcp/*` . Dùng Bearer token hoặc X-API-Key header.
7. `errorHandler` — global error boundary, trả về JSON 500 với trace.

2.3 MCP Server (src/server/mcpServer.ts)

Dùng @modelcontextprotocol/sdk v1.29.0.

- **Server identity:** name= kiro-backend-mcp , version= 1.0.0
- **Capabilities:** { tools: {} }
- **ListTools handler:** Chỉ trả về CORE tools (9 tools: mem_search , mem_ingest , mem_ingest_file , code_search , get_curated_context , find_tools , execute_dynamic_tool , orchestration_status , drawio_export_png , drawio_auto_layout). Non-core tools chỉ discoverable qua find_tools .
- **CallTool handler:** Lookup handler từ registry → execute → trackToolUsage() → broadcast notification nếu tool có prefix ingest/create/update/delete/tag.
- **Transport management:** registerTransport() / broadcastNotification() — gửi real-time events notifications/kb_entry_added , notifications/kb_entry_updated , notifications/kb_entry_deleted , notifications/tag_created tới tất cả connected transports.

2.4 Module System (src/modules/ModuleRegistry.ts)

Mỗi module implement interface:

```
interface IModule {
  readonly name: string;
  readonly status: ModuleStatus; // 'initializing' | 'ready' | 'error' | 'stopped'
  initialize(): Promise<void>;
  shutdown(): Promise<void>;
  getToolHandlers(): Map<string, ToolHandler>;
  getToolDefinitions(): ToolDefinition[];
}
```

Module	File	Tools	Chức năng
Memory	modules/memory/MemoryModule.ts	30+ mem_*	KB lưu trữ, search, CRUD, tagging, graph, scoring, evolution, session, audit, masking
CodeIntel	modules/code-intel/CodeIntelModule.ts	20+ code_* + drawio_*	AST parsing, symbol search, call/dependency/impact analysis, code context
Orchestration	modules/orchestration/OrchestrationModule.ts	4 tools	Quản lý child MCP server, find_tools semantic search, health monitoring, reconnect
Analytics	modules/analytics/AnalyticsModule.ts	2 tools	Quality scoring, analytics summary
KBGraph	modules/kb-graph/KBGraphModule.ts	3 tools	KB graph CRUD, spatial queries
Utility	modules/utility/UtilityModule.ts	2 tools	agent_log, stream_write_file

Web	modules/web/WebModule.ts	6 web_*	Internet tools với SSRF guard + rate limiter
-----	--------------------------	---------	--

Tool Routing Flow:

```
ToolRouter.route(request)
  → registry.getToolHandlers() // Map<tool_name, handler>
  → handler(args)              // direct function call
  → ToolResult { content, isError }
```

Tool Visibility Tiers (SA4E-18, src/config/CoreTools.ts):

- CORE_TOOLS (9 tools) — xuất hiện trong tools/list
- META_TOOLS (3 tools) — luôn visible: find_tools, execute_dynamic_tool, orchestration_status
- Tất cả tools khác — discoverable qua find_tools (semantic search trên mcp_tools table với vector embeddings)

3. Knowledge Base — Storage, Scope, Masking & Evolution

3.1 Database Schema

knowledge_entries (21 columns)

```
CREATE TABLE knowledge_entries (
  id          TEXT PRIMARY KEY,          -- UUID v4
  tenant_id   TEXT NOT NULL DEFAULT 'default',
  title       TEXT NOT NULL,
  content     TEXT NOT NULL,
  entry_type  TEXT NOT NULL DEFAULT 'note', -- note, decision, spec, bugfix, pattern,
reference
  source      TEXT,                      -- origin (e.g. "KSA-14", "chat", "code-
review")
  tags        TEXT,                      -- JSON array của tags
  confidence  REAL DEFAULT 1.0,          -- 0.0 - 1.0
  superseded_by TEXT,                    -- id của entry mới thay thế entry này
  supersedes  TEXT,                      -- JSON array các entry bị entry này thay thế
  version     INTEGER DEFAULT 1,
  metadata    TEXT,                      -- JSON object linh hoạt
  embedding   BLOB,                      -- 384-dim Float32 vector (ONNX all-MiniLM-L6-
v2)
  scope       TEXT NOT NULL DEFAULT 'user', -- user | project | shared
  user_id     TEXT,                      -- owner userId
  project_id  TEXT,                      -- project context
  created_at  TEXT NOT NULL DEFAULT (datetime('now')),
  updated_at  TEXT NOT NULL DEFAULT (datetime('now')),
  last_accessed TEXT,
  access_count INTEGER DEFAULT 0,
  outcome_score REAL,                    -- ghi nhận từ OutcomeService
  outcome_data TEXT                      -- JSON: { iterations, verified, accuracy }
);
```

knowledge_fts (FTS5 virtual table)

```
CREATE VIRTUAL TABLE knowledge_fts USING fts5(  
  title, content, tags,  
  content=knowledge_entries, content_rowid=rowid,  
  tokenize='porter unicode61'  
);
```

Sync qua 3 triggers: `knowledge_ai` (AFTER INSERT), `knowledge_ad` (AFTER DELETE), `knowledge_au` (AFTER UPDATE). Update trigger dùng `INSERT OR REPLACE` thay vì delete+insert.

knowledge_graph_edges

```
CREATE TABLE knowledge_graph_edges (  
  id TEXT PRIMARY KEY,  
  source_entry_id TEXT NOT NULL REFERENCES knowledge_entries(id),  
  target_entry_id TEXT NOT NULL REFERENCES knowledge_entries(id),  
  relationship TEXT NOT NULL, -- relates_to, depends_on, supersedes,  
  implements, contradicts  
  weight REAL DEFAULT 1.0,  
  metadata TEXT,  
  created_at TEXT DEFAULT (datetime('now'))  
);
```

knowledge_vectors

Bảng riêng cho vector search (384-dim Float32), đồng bộ với callback lifecycle.

memory_sessions

```
CREATE TABLE memory_sessions (  
  id TEXT PRIMARY KEY,  
  session_name TEXT,  
  user_id TEXT,  
  project_id TEXT,  
  started_at TEXT NOT NULL DEFAULT (datetime('now')),  
  ended_at TEXT,  
  metadata TEXT  
);
```

memory_audit

```
CREATE TABLE memory_audit (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  session_id TEXT,  
  action TEXT NOT NULL, -- SESSION_START, SESSION_END, PROMOTE, DEMOTE, INSERT,  
  UPDATE, DELETE  
  entity_type TEXT,  
  entity_id TEXT,
```

```

    details      TEXT,
    user_id      TEXT,
    created_at   TEXT DEFAULT (datetime('now'))
);

```

3.2 Scope Isolation (IsolationLayer.ts)

Ba cấp scope: USER → PROJECT → SHARED (chỉ promotion 1 chiều, không demotion ngược).

buildReadFilter() — injected vào mọi `mem_search` , `mem_get` , `mem_list` :

```

buildReadFilter(userId, projectId, scope): SQLFilter {
  if (scope === 'user')    → WHERE user_id = :userId
  if (scope === 'project') → WHERE project_id = :projectId
  if (scope === 'shared')  → WHERE scope = 'shared'
  // Kết hợp: user thấy entries của họ + entries project + shared
  → WHERE (user_id = :userId AND scope = 'user')
      OR (project_id = :projectId AND scope = 'project')
      OR scope = 'shared'
}

```

buildWriteDecorator() — tự động gắn `__projectId` , `__userId` , `__workspaceRoot` trước khi write. Client không bao giờ tự supply scope keys.

validateMutationOwnership() — kiểm tra user có quyền mutate entry không:

- User scope: chỉ owner
- Project scope: thành viên project
- Shared scope: cần permission đặc biệt (PROMOTE)

ScopePromotionService — chạy cycle mỗi 60 phút, promote entries từ USER lên PROJECT hoặc SHARED dựa trên:

- Access count > threshold
- Confidence > 0.8
- Được promote bởi người dùng có quyền

3.3 Sensitive Data Masking

Hai lớp detector, áp dụng ở read-time (không modify dữ liệu gốc):

PiiDetector (masking/detectors/PiiDetector.ts)

5 regex patterns, fail-open (cho phép đọc, chỉ mask):

Pattern	Target	Regex
email	Địa chỉ email	<code>[\\w._%+-]+@[\\w.-]+\\. [a-zA-Z]{2,}</code>
phone	Số điện thoại	<code>\\+?\\d{1,4}[\\s-]?\\((?\\d+\\)\\)?[\\s-]?\\d+[\\s-]?\\d+[\\s-]?\\d+</code>
ssn	SSN Mỹ	<code>\\b\\d{3}-\\d{2}-\\d{4}\\b</code>
credit_card	Thẻ tín dụng	<code>\\b\\d{4}[-]?\\d{4}[-]?\\d{4}[-]?\\d{4}\\b</code>
ip_address	IPv4	<code>\\b\\d{1,3}\\.\\d{1,3}\\.\\d{1,3}\\.\\d{1,3}\\b</code>

CredentialDetector (`masking/detectors/CredentialDetector.ts`)

5 patterns, **fail-closed** (nếu match, trả về lỗi thay vì mask — credentials không bao giờ được expose):

Pattern	Target	Hành vi
api_key	API keys dạng sk-..., pk-...	Fail-closed
jwt	JWT tokens (3 base64 segments)	Fail-closed
password	"password=", "passwd="	Fail-closed
connection_string	ODBC/JDBC connection strings	Fail-closed
private_key	-----BEGIN ... PRIVATE KEY-----	Fail-closed

Role-based visibility (MaskingMiddleware.ts):

- ADMIN : không mask gì cả (trừ credentials)
- DEVELOPER : mask PII ở chế độ `partial` (giữ prefix)
- USER : mask PII full
- EXTERNAL : mask PII + business logic rules

3.4 Evolution System

CompositeScorer (`evolution/CompositeScorer.ts`)

Kết hợp 5 strategies với trọng số khác nhau:

```
compositeScore = (  
  temporalScore * 0.15 +  
  confidenceScore * 0.30 +  
  supersededScore * -0.25 + (penalty)  
  outcomeScore * 0.20 +  
  predictiveScore * 0.10  
) * 100
```

Strategy	File	Công thức	Đặc điểm
Temporal	TemporalStrategy.ts	$0.5^{(ageDays/halfLifeDays)}$	Half-life mặc định 30 ngày
Confidence	ConfidenceStrategy.ts	<code>confidence * 100</code>	Dùng trực tiếp confidence field
Superseded	SupersededStrategy.ts	<code>superseded_by != null → -50</code>	Kiến thức bị thay thế bị phạt nặng
Outcome	OutcomeStrategy.ts	<code>avg(verifiedRatio) * 100</code>	Dựa trên outcome_data JSON
Predictive	PredictiveStrategy.ts	Weighted recency trend	10 outcomes gần nhất, thời gian gần hơn có weight cao hơn

DecayService (`evolution/DecayService.ts`)

Chạy batch-style (100 entries/batch) trên entries có `last_accessed > decay interval`:

```
newConfidence = MAX(confidence * (1 - decayRate), confidenceFloor)
```


- `decayRate` : 0.05 (mặc định)
- `confidenceFloor` : 0.1 (không decay dưới mức này)

Scheduler (`evolution/Scheduler.ts`)

- **Decay interval**: configurable, mặc định 24h
- **Stagnation detection**: 6h cycle — entries không có `last_accessed` thay đổi trong 2 cycle bị đánh dấu stagnation
- **Epoch trigger**: `mem_epoch_trigger` tool — force scoring cycle
- **Stagnation report tool**: `mem_stagnation_report` — list entries stagnated

OutcomeService

Ghi nhận kết quả thực tế khi user verify một entry:

- `mem_record_outcome(entryId, verified: boolean, accuracy: number)`
- Update `outcome_score` và `outcome_data` trong `knowledge_entries`
- Ảnh hưởng tới OutcomeStrategy scoring

3.5 Memory Module Initialization Detail

Khi `MemoryModule.initialize()` chạy:

1. `loadConfig()` → `DatabaseManager` → SQLite WAL mode
2. Run migrations: 001 (scope columns), 002 (evolution columns), 003 (pending tasks)
3. Tạo `SqliteDbAdapter` → `MemoryEngine` (extends `MemoryEngineCrud`)
4. Start session (`startSession`) với tên unique `kiro-backend-{pid}`
5. Tạo `QueryLayer` → `MemoryToolDispatcher`
6. Wire `ConvertToolResolver` qua `RegistryOrchestrationGateway` (dynamic tool discovery)
7. Khởi tạo `TaskWorker` — background processor với:
 - `baseInterval` : 2s, `maxInterval` : 30s, `staleThreshold` : 300s, `maxRetries` : 3
 - `recoverStaleTasks()` → `start()`
8. Khởi tạo `ScopePromotionService` — chạy promotion cycle mỗi 60 phút
9. Khởi tạo LLM health check (background, fire-and-forget):
 - Gọi `/v1/models` endpoint với timeout 5s
 - Nếu reachable → tạo `LLMService` + `TagAnalyzerService` + `ClassifyService`
 - `TagAnalyzerService` : LLM auto-tagging khi ingest
 - `ClassifyService` : Smart KB Ingest (classify trước khi lưu)
10. Start evolution scheduler (`startScheduler`)

4. User Management & API Security

4.1 Admin Database Schema (`admin/db/schema.ts`)

8 tables cho user & access management:

users:

```
CREATE TABLE users (
  id          TEXT PRIMARY KEY,
  username    TEXT UNIQUE NOT NULL,
  email       TEXT UNIQUE NOT NULL,
  password_hash TEXT NOT NULL,      -- bcrypt
```

```

display_name    TEXT,
avatar_url      TEXT,
status          TEXT DEFAULT 'active',  -- active, inactive, suspended
role            TEXT DEFAULT 'admin',   -- admin, super_admin
force_password_change INTEGER DEFAULT 0,
last_login      TEXT,
metadata        TEXT,                  -- JSON
created_at      TEXT DEFAULT (datetime('now')),
updated_at      TEXT DEFAULT (datetime('now'))
);

```

access_groups:

```

CREATE TABLE access_groups (
  id          TEXT PRIMARY KEY,
  name        TEXT UNIQUE NOT NULL,
  description TEXT,
  is_system   INTEGER DEFAULT 0,
  created_at  TEXT DEFAULT (datetime('now'))
);

```

group_permissions:

```

CREATE TABLE group_permissions (
  id          TEXT PRIMARY KEY,
  group_id     TEXT NOT NULL REFERENCES access_groups(id) ON DELETE CASCADE,
  permission_id TEXT NOT NULL,
  resource_type TEXT,
  resource_id  TEXT,                  -- NULL = all resources
  role_data    TEXT,                  -- JSON: additional role config
  created_at   TEXT DEFAULT (datetime('now'))
);
-- UNIQUE(group_id, permission_id, resource_type, resource_id)

```

sessions:

```

CREATE TABLE sessions (
  id          TEXT PRIMARY KEY,          -- opaque hex token
  user_id     TEXT NOT NULL REFERENCES users(id),
  token_type  TEXT DEFAULT 'session',   -- session, api_key
  device_info TEXT,
  ip_address  TEXT,
  expires_at  TEXT NOT NULL,
  last_activity TEXT DEFAULT (datetime('now')),
  created_at  TEXT DEFAULT (datetime('now'))
);

```

- Token expiry: 24h (configurable)
- Device + IP tracking
- `validateSession()` kiểm tra active/expired/inactive user

audit_log: ghi mọi hành động admin (login, CRUD user, permission change, config change)

4.2 RBAC Middleware (admin/middleware/rbac.middleware.ts)

Route → permission mapping (25+ routes):

Route Pattern	Required Permission
GET /api/admin/dashboard/*	DASHBOARD_VIEW
GET /api/admin/kb/entries	KB_READ
POST /api/admin/kb/entries	KB_WRITE
POST /api/admin/kb/promote	KB_PROMOTE
GET /api/admin/users	USER_VIEW
POST /api/admin/users	USER_MANAGE
GET /api/admin/rbac/*	RBAC_VIEW
POST /api/admin/rbac/*	RBAC_MANAGE
GET /api/admin/config	CONFIG_VIEW
POST /api/admin/config	CONFIG_MANAGE
GET /api/admin/mcp/*	MCP_VIEW
POST /api/admin/mcp/*	MCP_MANAGE

Flow: JWT extracted → group_permissions lookup → check permission trên route → allow/deny

4.3 JWT Authentication (middleware/jwt-auth.ts)

- **Algorithm:** HS256
- **Secret:** từ config (CODE_INTEL_JWT_SECRET env, fallback auto-generated)
- **Claims structure:**
 - sub : userId
 - wid : workspaceRoot hash
 - pid : projectId (primary)
 - pids : JSON array các projectIds user có quyền
 - iat , exp
- **ProjectContext injection:** validateJwtConfig() → projectContext = createProjectContext(sub, wid, pid, pids) → gắn vào Hono context
- **Anonymous fallback:** nếu không có Authorization header và CODE_INTEL_REQUIRE_AUTH không bật → tạo anonymous context

Admin session token: opaque hex string, stored in sessions table. Validated by looking up the session record + checking expiry.

4.4 SEC-02 / SEC-03 Enforcement (server/routes/tools.ts)

Ba hàm bảo vệ scope key injection:

1. `stripReservedKeys(args)` — xoá `__projectId` , `__userId` , `__workspaceRoot` nếu client gửi lên (không tin client)
2. `stampUserId(args, userId)` — gán `__userId` từ server context
3. `stampProjectContext(args, projectContext)` — gán `__projectId` , `__workspaceRoot` từ server context

Flow: request → stripReservedKeys → validate args (Zod) → stampUserId → stampProjectContext → handler

4.5 PKCE OAuth (extension/src/auth/PkceService.ts)

Cho SSO flow:

- Generate S256 `code_verifier` (random 128 bytes) → `code_challenge` (base64url SHA256)
- Authorization URL với `code_challenge`
- Token exchange: gửi `code_verifier` → backend verify
- State parameter chống CSRF

4.6 API Key Authentication (middleware/api-key-auth.ts)

- **Header:** `Authorization: Bearer <key>` hoặc `X-API-Key: <key>`
- **Scope:** only `/api/tags/*` và `/mcp/*`
- Fixed key từ config (admin-generated)

5. Multi-IDE Agent Deployment

5.1 Strategy Pattern (extension/src/ide-adapters.ts)

6 IDE targets, mỗi target có adapter riêng:

```
interface IdeAdapter {
  name: string;
  agentDir: string;
  steeringDir: string;
  hooksDir: string;
  additionalFiles?: { path: string; content: string }[];
}
```

IDE	Agents Dir	Steering Dir	Adapter Class
Kiro (default)	.kiro/agents/	.kiro/steering/	KiroAdapter
VS Code / Copilot	.github/agents/	.github/copilot-instructions.md	PreConvertedAdapter("github-copilot")
Claude Code	.claude/agents/	.claude/rules/ + CLAUDE.md	PreConvertedAdapter("claude-code")
OpenAI Codex	AGENTS.md + agents/	—	PreConvertedAdapter("codex-openai")
OpenCode	.opencode/agents/	.opencode/skills/ + opencode.json	PreConvertedAdapter("opencode")
AntiGravity	.agents/	skills/	PreConvertedAdapter("antigravity")

5.2 4 Core Components (extension/src/config.ts)

```
CORE_COMPONENTS = {
  agents: { source: "resources/.kiro/agents/", target: ".kiro/agents/" },
  steering:{ source: "resources/.kiro/steering/", target: ".kiro/steering/" },
  hooks: { source: "resources/.kiro/hooks/", target: ".kiro/hooks/" },
  templates:{source: "documents/templates/", target: "documents/templates/" }
}
```

- **Agents:** 9 agent prompts (BA, SA, QA, DEV, DevOps, UI, Security, SM, TA) — SM prompt dài nhất (~52KB)
- **Steering:** 41+ steering files cho hướng dẫn agent behavior
- **Hooks:** Code index + drawio validation hooks
- **Templates:** BRD, FSD, TDD, STP, STC, DPG, RLN, UG templates

5.3 Injection System (extension/src/injector.ts)

4 operations:

1. `injectAll(target: IdeTarget)` — inject tất cả components + MCP bundling
2. `injectSelective(components: string[], target: IdeTarget)` — pick component list
3. `update(target: IdeTarget)` — update lên version mới nhất với conflict resolution (backup/overwrite/skip)
4. `injectComponent(name: string, source: string, targetDir: string)` — single component

InjectorHelpers (extension/src/injector-helpers.ts):

Function	Chức năng
forceUpdate(src, dest)	Ghi đè không hỏi
updateSkipModified(src, dest, manifest)	Bỏ qua file đã modified bởi user
updateWithBackup(src, dest, backupSuffix)	Backup trước khi ghi đè

5.4 MCP Variants

3 cách chạy backend MCP, tùy theo môi trường:

Variant	Command	Đặc điểm
Python (uvx)	uvx kiro-mcp	Recommended, nhanh nhất, zero setup
Node.js (npx)	npx kiro-mcp	Fallback, cần Node.js
Kotlin (JAR)	java -jar kiro-mcp.jar	Download JAR, cần JVM

5.5 Pre-converted Files

Thư mục `conversions/` chứa agents/steering/hooks đã convert sẵn cho từng IDE target:

```
conversions/
  opencode/      → .opencode/agents/*, opencode.json, .opencode/skills/*
  claude-code/   → .claude/agents/*, CLAUDE.md, .claude/rules/*
```

```
github-copilot/ → .github/agents/*, .github/copilot-instructions.md
codex-openai/   → AGENTS.md, agents/*
antigravity/    → .agents/*, skills/*
```

Mỗi target folder có CORE_COMPONENTS tương ứng (agents/, steering/, hooks/).

5.6 Checksum & Version Tracking (extension/src/checksum.ts)

Quản lý phiên bản component đã inject:

- **Manifest:** JSON file .kiro/manifest.json lưu checksums của từng file đã inject
- **detectModifiedFiles(manifest)** : so sánh checksum hiện tại với manifest → xác định file nào đã bị user sửa (modified) vs file nào còn nguyên (pristine)
- **buildManifestAfterInject(targetDir)** : tạo manifest mới sau khi inject
- **Version tracking:** mỗi component có version string, so sánh để quyết định có cần update không

5.7 Platform Swap (extension/src/platform-swap/)

Cho phép chuyển đổi giữa các IDE target mà không mất cấu hình:

```
PlatformDetector → BackupManager (backup old config)
→ SwapExecutor (remove old → inject new)
→ StateManager (persist current platform)
→ PlatformStatusBar (indicator trên status bar)
```

6. Chat Window & Agentics Pipeline

6.1 Chat Panel Architecture

ChatPanelProvider (extension/src/chat-panel/chat-panel-provider.ts) — webview-based chat interface:

- **Multi-tab** (ChatStateManager): nhiều conversation tabs, mỗi tab có state riêng
- **Streaming responses** (StreamHandler): real-time token streaming via `handleStreamEvent`
- **Model selector:** auto, Claude Sonnet/Haiku, GPT-4o, Ollama, Kiro, OpenRouter, LM Studio
- **Autopilot/Supervised modes:** Autopilot tự động route intent, Supervised chờ human approval
- **Context picker:** files, folders, problems, git diff, terminal, spec, currentFile, steering, MCP
- **Attachment support:** file picker gửi kèm
- **Code apply:** replace selection hoặc entire file
- **Code insert:** insert at cursor hoặc new file
- **Token counting** (ContextUsageTracker)
- **Pipeline graph visualization** (graph-viz.js)
- **Approval request handling:** approve/reject/cancel với feedback text

6.2 Message Routing (extension/src/chat-panel/message-routing.ts)

5 loại message, xử lý theo thứ tự ưu tiên:

```
routeUserMessage(text, enrichedText, getEngine, sendToWebview)
```

1. Direct commands (exact match, case-insensitive):

Command	Handler
---------	---------

status	getEngine().getCurrentNodeStates() → graphUpdate
resume	getEngine().resume(threadId) — resume paused pipeline
cancel	getEngine().cancel()

2. Agent commands (pattern: /agent-name task description):

- Match: `/^([a-z][-a-z]*)\s+(.+)$/i`
- Gọi `getEngine().invokeChat(\ [Agent: ${agentName}] ${agentTask})``
- Ví dụ: `/sm-agent KSA-14 full`

3. Ticket pattern (pattern: TICKET-KEY action):

- Match: `/^([A-Z]+[-\d+])\s+(.+)$/`
- Parse action keywords → phase mapping:

Keyword	SDLCPhase
brd, tao brd	requirements
fsd, tao fsd	specification
tdd, tao tdd	design
stp, tao stp	test_planning
implement	implementation
test	testing
deploy	deployment
full	all

- Gọi `getEngine().invoke(ticketKey, phase, enrichedText)`

4. Chat (fallback):

- Gọi `getEngine().invokeChat(enrichedText)`
- Fire agentStop hooks sau khi chat hoàn tất

Context enrichment: `buildEnrichedText(text, context)` — chèn context blocks được chọn từ context picker vào dạng XML tags:

```
<context>
  <file name="main.ts">
    ...content...
  </file>
  <git_diff name="working tree">
    ...diff...
  </git_diff>
</context>
```

User message here

6.3 LangGraph Engine (extension/src/langgraph/engine/langgraph-engine.ts)

Pipeline orche engine dạng **StateGraph** (LangGraph.js):

```
class LangGraphEngine {
  graph?: StateGraph<typeof PipelineAnnotation>;
  activePipeline?: PipelineInvocation;
  hookEngine: HookEngine;
  streamHandler: StreamHandler;
  llmProvider: LlmProvider;

  // Core API
  async invoke(ticketKey, phase, chatInput)
  async invokeChat(chatInput)
  async resume(threadId)
  cancel()
  handleApproval(threadId, decision, feedback)
  listPersistedPipelines()
  getCurrentNodeStates()
  handleLiveSpecMutation(newSpec)
}
```

Pipeline state (langgraph/core/state.ts — PipelineAnnotation, 65+ annotated fields):

Field	Type	Reducer	Default
ticketKey	string	—	—
threadId	string	—	—
currentPhase	SDLCPhase	—	—
intent	PipelineIntent	replace	"chat"
pipelineStatus	PipelineStatus	—	—
documents	Record<string, DocumentState>	—	{}
agentOutputs	AgentOutput[]	replace	[]
chatHistory	ChatMessage[]	append (slice 200)	[]
errors	PipelineError[]	—	—
retryCount	Record<string, number>	—	—
qualityGateResults	Record<string, QualityGateResult>	merge	{}
toolCalls	LlmToolCall[]	replace	null
toolResults	Array<{toolCallId, name, content}>	append	[]
agentScratchpad	LlmMessage[]	replace	[]
agentIterations	number	replace	0

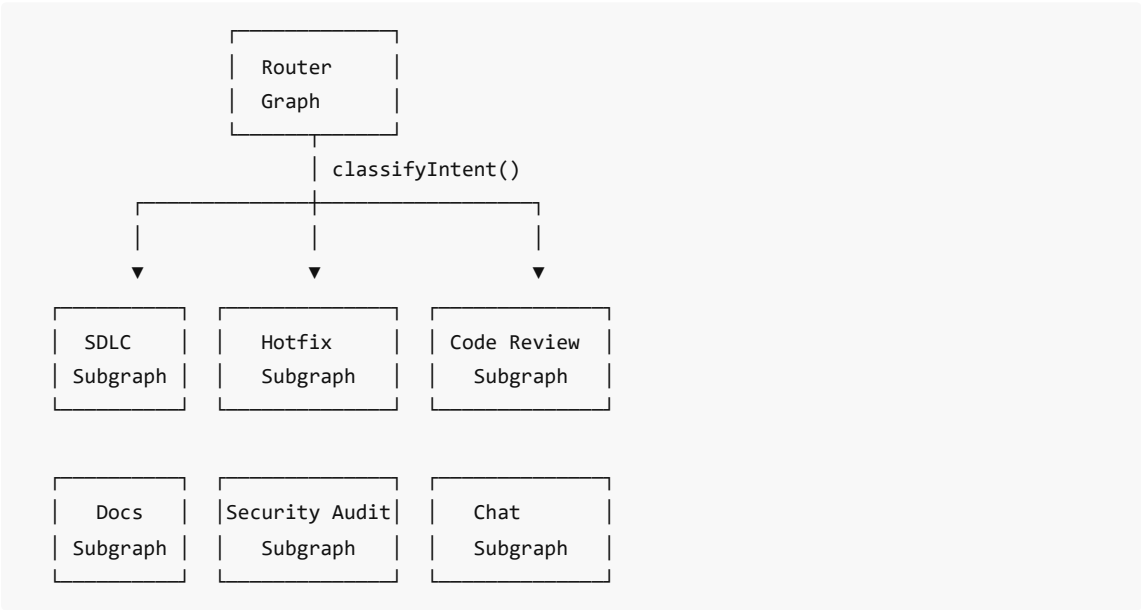
verifyPassed	boolean	replace	true
verifyAttempts	Record<string, number>	merge	{}
activeStrategy	Record<string, string>	merge	{}
strategyHistory	StrategyEvent[]	append (slice 20)	[]
autonomyLevel	AutonomyLevel	replace	"L2"
analyzedIntent	IntentAnalysis	replace	null
currentPhaseIndex	number	replace	0
pipelineDefinition	PipelineDefState	replace	null

6.4 Router Graph & 6 Subgraphs (extension/src/langgraph/router/router-graph.ts)

Router Graph — node đầu vào, dùng `classifyIntent()` để xác định intent:

```
async function buildRouterGraph(mcpBridge, streamHandler, checkpointer, llmProvider?,
hookEngine?)
```

Intent Classification → chọn 1 trong 6 subgraphs:



Subgraph	Import	Khi nào dùng
sdlc-graph	pipeline/sdlc-graph.ts	Intent="sdlc" — multi-phase pipeline qua 9 agents
hotfix-graph	subgraphs/hotfix-graph.ts	Intent="hotfix" — hotfix nhanh, skip quality gates
code-review-graph	subgraphs/code-review-graph.ts	Intent="code_review" — review code
docs-graph	subgraphs/docs-graph.ts	Intent="docs" — generate documentation

security-audit-graph	subgraphs/security-audit-graph.ts	Intent="security_audit" — security assessment
chat-graph	subgraphs/chat-graph.ts	Default — conversation với LLM

Lazy loading: subgraphs chỉ được import khi intent tương ứng được chọn, dùng dynamic `import()`. Kết quả được cache trong `subgraphCache` Map.

Thread ID: configurable khi invoke, dùng cho `WorkspaceCheckpoint`er.

6.5 Pipeline Execution Flow

SDLC Subgraph (pipeline/sdlc-graph.ts) — multi-phase pipeline:

```
analyze_input
  → route_to_agent (dựa trên currentPhaseIndex)
  → agent_node (gọi MCP tools tùy phase)
  → verify_output (quality gate)
  → human_approval (nếu approvalRequired)
  → advance_phase
  → [loop] route_to_agent → ... → complete
```

Mỗi phase tương ứng một agent:

- `requirements` → BA agent
- `specification` → TA agent (review) + SA agent (TDD)
- `test_planning` → QA agent
- `implementation` → DEV agent
- `testing` → QA agent
- `deployment` → DevOps agent

Tool proxy: MCP tool calls từ agent → `LangGraphEngine` → `McpBridge` → `WrapperServer` (port 9181) → `RemoteBackendClient` → Backend `/api/tools/execute`.

6.6 WorkspaceCheckpoint (extension/src/langgraph/core/checkpointer.ts)

Persist pipeline state xuống workspace file:

- **Storage:** `.kiro/pipelines/{threadId}.json`
- **Triggers:** sau mỗi node execution (checkpoint tại mọi graph step)
- **Content:** full `PipelineAnnotation` state (JSON serialized)
- **Resume flow:** `resume(threadId)` → load state → reconstruct graph → continue từ `resumePoint`
- **Use cases:** VS Code restart, pipeline pause/resume, crash recovery

6.7 HookEngine (extension/src/langgraph/hooks/hook-engine.ts)

Sandboxed hook system cho pipeline events:

Hook	Execution Time	Mục đích
agentStart	Trước khi agent chạy	Validate input, inject context
agentStop	Sau khi agent hoàn tất	Post-process output, trigger notifications

toolCall	Trước khi tool call	Modify args, check permissions
toolResult	Sau khi tool trả về	Transform result, log usage
error	Khi có lỗi	Error recovery, fallback strategies

Hooks được load từ `.kiro/hooks/` (injected component), chạy trong sandbox với API giới hạn.

6.8 SDLC Pipeline Agent Prompts

Các agent prompts được inject vào `.kiro/agents/` (9 files, format Markdown):

Agent	File	Vai trò
BA	ba-agent.md	Business Analyst — gather requirements, viết BRD
TA	ta-agent.md	Technical Architect — review FSD, enrich technical depth
SA	sa-agent.md	Solution Architect — thiết kế TDD
QA	qa-agent.md	QA Engineer — STP, STC, verify
DEV	dev-agent.md	Developer — implement code từ TDD
DevOps	devops-agent.md	DevOps — Deployment Guide, CI/CD, Release Notes
UI	ui-agent.md	UI/UX Designer — mockup, wireframe
Security	security-agent.md	Security Engineer — security audit
SM	sm-agent.md	Scrum Master — điều phối pipeline (prompt lớn nhất ~52KB)

SM Agent Prompt (`resources/.kiro/agents/prompts/sm-agent.md`):

- Phân tích ticket, xác định scope
- Routing task tới agent phù hợp
- Theo dõi tiến độ qua các phase
- Quality gate verification (6 gates)
- Loop prevention (circuit breaker, max iterations)

6.9 Agent Pipeline Extraction

`agentRegistry.loadPipeline()` (`extension/src/langgraph/agents/registry.ts`):

- Đọc agents từ `.kiro/agents/*.md`
- LLM-based pipeline extraction: best-effort phân tích prompt để xác định các phase và dependencies
- Fallback: static phase list (requirements → specification → design → test_planning → implementation → testing → deployment)
- `pipelineDefinition` lưu vào state

6.10 Agentic Features

AutopilotMode:

- **L0 (Manual):** user chọn từng phase
- **L1 (Supervised):** pipeline tự động advance nhưng chờ human approval mỗi phase

- **L2 (Autopilot):** pipeline chạy hoàn toàn tự động, chỉ hỏi khi cần clarification

Live Spec Mutation (`handleLiveSpecMutation()`):

- Sandboxed hot-swap specification giữa pipeline
- 3 validation rules: non-empty, valid agent names, no LLM errors

Human-in-the-loop: `handleApproval(threadId, decision, feedback)` :

- `approve` : continue pipeline
- `reject` : pause pipeline, include user feedback
- `approve_with_changes` : approve + feedback (modify spec)

7. Data Flow: Từ `@sm-agent KSA-14` đến kết quả

7.1 Flow chi tiết: User message → Chat → Pipeline

1. User gõ "KSA-14 full" trong Chat Panel
2. `ChatPanelProvider.onDidReceiveMessage()`
→ `messageHandler.handleUserMessage(text)`
3. `message-routing.ts`:
 - a. `routeUserMessage("KSA-14 full")`
 - b. Match ticket pattern: `/^([A-Z]+-\d+)\s+(.+)$/`
→ `ticketKey="KSA-14", action="full"`
 - c. `parsePhase("full")` → "all"
 - d. `LangGraphEngine.invoke("KSA-14", "all", enrichedText)`
4. `LangGraphEngine.invoke()`:
 - a. `ensureGraph()` → `buildPipelineGraph()`
 - b. `classifyIntent("KSA-14 full")` → "sdlc"
 - c. `RouterGraph` → lazy-load SDLC subgraph
 - d. `graph.invoke(initialState)` → pipeline nodes:
 - i. `analyze_input`: set `currentPhase="requirements"`
 - ii. `route_to_agent`: select BA agent
 - iii. `ba_agent`: BA calls tools (`mem_search`, `fetch_url`, ...)
→ MCP tool call → `LangGraphEngine.McpBridge`
→ `WrapperServer` (port 9181) → REST POST `/api/tools/execute`
→ Backend `ToolRouter` → `MemoryModule` handler
→ `MemoryEngine.search()` → `SQLite FTS5` + `CompositeScorer`
→ stream response → `StreamHandler` → chat panel
 - iv. `verify_output`: quality gate check
 - v. `human_approval`: nếu `AutopilotMode=L1` (Supervised)
 - vi. `advance_phase`: `currentPhaseIndex++`
 - vii. Lặp cho TA, SA, QA, DEV, ... phases
5. Mỗi phase output:
 - `agentOutputs[]` appended
 - `documents/{KSA-14}/` file created
 - stream event → Chat Panel real-time update
 - `graph-viz.js` node status update
6. Hoàn tất → `pipelineStatus="completed"`
→ `WorkspaceCheckpointinter.save()` → `.kiro/pipelines/{threadId}.json`

7.2 Tool Call Flow (chi tiết)

```
LLM Agent → MCP tools/call
  → LangGraphEngine (processToolCalls)
  → McpBridge → WrapperServer /mcp (tools/call)
  → routeToolCall():
    local? → stream_write_file, embed_image (handle locally)
    remote? → callWithProxy():
      → Base64ProxyService.proxyInput(file_path→content_base64)
      → RemoteBackendClient.restCallTool()
      → POST /api/tools/execute (JWT + X-Project-Id headers)
      → Backend: jwtAuth → ToolValidator.validate()
      → stripReservedKeys() → Zod schema validation
      → stampUserId() → stampProjectContext()
      → ToolRouter.route(toolName) → module handler
      → MemoryEngine / CodeIntel handler
      → Result → stampProjectContext → response
      → Base64ProxyService.proxyOutput(output_base64→file_path)
      → WrapperServer response → McpBridge
      → toolResults[] → next LLM call
```

7.3 Chat-Only Flow

```
User message (no ticket match)
  → routeUserMessage fallback: LangGraphEngine.invokeChat(text)
  → RouterGraph → classifyIntent → "chat"
  → lazy-load chat-graph.ts
  → buildChatSubgraph(streamHandler, llmProvider, mcpBridge, wsRoot, hookEngine)
  → LLM conversation with tool access
  → chatHistory[] auto-maintained (last 200 messages)
  → agentStop hook fire after completion
```

8. Design Patterns

Pattern	Ví dụ
Strategy	DatabaseAdapter (SqliteAdapter/PostgresAdapter/MysqlAdapter), IdeAdapter (KiroAdapter/PreConvertedAdapter)
Facade	MemoryEngine (facade over DatabaseAdapter), McpClientManager (facade coordinating HealthMonitor + ReconnectManager + ConnectionStateTracker)
Factory	WebviewPanelManager.createPanel(), DatabaseAdapterFactory.create()
Singleton	EmbeddingService.getInstance(), ModuleRegistry, remote-backend-client
Observer	onStatusChange (MCP), onStateChange (Auth), onNotification (KbEventBus), EventEmitter pattern
State Machine	AuthManager (3 states), ConnectionManager (3 states), ConnectionStateTracker (6 states)

Proxy	RemoteBackendClient (proxies MCP calls), Base64ProxyService (file↔base64)
Module	IModule interface → 7 concrete modules
Composite	CompositeScorer (5 scoring strategies)
Template Method	BasePanel (abstract getHtml/loadData/handleMessage)
Command	CommandRegistrar (23 commands)
Chain of Responsibility	Middleware stack (security → bodyLimit → logger → rateLimiter → jwtAuth → apiKeyAuth)
Lazy Loading	Subgraph import-on-demand trong RouterGraph
Sandbox	HookEngine (limited API), LiveSpecMutation (3 validation rules)
Checkpoint	WorkspaceCheckpointner (pipeline state persistence)

9. File & Test Statistics

Component	Source files	Test files	Notes
Backend <code>src/</code>	~200 files	~66 test files	570 tests (v1.11.0)
Extension <code>src/</code>	~140 files	E2E + unit + integration	Vitest + Mocha + Sinon
Backend <code>database/</code>	5 adapters + factory + config + migration	Included in backend tests	
Backend <code>electron/</code>	15+ files	-	Desktop app
Backend <code>servers/nextjs</code>	30+ files	Cypress component tests	Presentation server
Admin viewer	20+ files	-	SPA frontend
Total	~400+ source files	600+ tests	